

SYRMA SGS TECHNOLOGY LIMITED

IT — SAP Analytics Platform

Setup & User Guide

GR/IR Reconciliation & Procurement Analytics Dashboard

Document Version 2.0 · June 2026

Table of Contents

1. Overview & What This Application Does
2. Required Software — Complete List
3. How to Open a Terminal / Command Prompt
4. Recommended Code Editors (IDEs)
5. Cloning the Project from Git
6. Backend Setup (Python / Flask)
7. Frontend Setup (Node.js / React)
8. Running the Application — Daily Usage
9. Using the Dashboard — Step-by-Step
10. AI Chat Assistant (Ollama — Optional)
11. Troubleshooting Common Errors
12. Quick-Reference Cheat Sheet

1. Overview & What This Application Does

The Syrma SGS Analytics Dashboard is a full-stack web application that runs entirely on your local machine (no internet required after setup). It processes SAP export files (CSV / Excel) to produce live, interactive analytics for:

- GR/IR (Goods Receipt / Invoice Receipt) Reconciliation
- Purchase Order spend analysis, supplier concentration, and aging
- Multi-currency financial reporting (auto-converted to INR)
- AI-powered chat assistant for natural-language data queries (optional)

The application has two parts that must both be running at the same time:

- Backend — a Python Flask server (runs on port 5000)
- Frontend — a React web app served by Vite (opens in your browser at port 5173)

□ **Think of it like this:**

The Backend is the engine that processes your data. The Frontend is the dashboard in your browser that displays it. Both must be started before you open the browser.

2. Required Software — Complete List

Install the following software before attempting to set up the project. Each item includes where to download it and what it is used for.

Software	Version	What it does
Git	Latest	Clone the project from GitHub / internal repo
Python	3.10 – 3.13	Run the Flask backend server
Node.js + npm	18+ (LTS)	Run the React frontend dev server
pip	Bundled with Python	Install Python packages automatically
A Terminal / Shell	Built-in	Type commands to start the app
A Web Browser	Any modern	View the dashboard (Chrome / Edge recommended)
Ollama (optional)	Latest	Run an AI chat assistant locally — NOT required to use the dashboard

□ **Important:**

Install Git, Python, and Node.js before anything else. After installing Python, close and reopen your terminal so it recognises the new commands. Do the same after installing Node.js.

3. How to Open a Terminal / Command Prompt

A terminal (also called a "command prompt" or "shell") is a text window where you type commands to install software and start the application. You do not need any coding knowledge to use it — you simply copy and paste the commands shown in this guide.

Option A — Windows PowerShell (Recommended)

PowerShell is the modern, powerful shell built into Windows 10 and 11.

- Press the `Windows` key on your keyboard.
- Type: `PowerShell` and press `Enter`.
- A blue window opens — this is your terminal. You are ready to type commands.

Alternative shortcut:

- Right-click the Start button → select "Windows PowerShell" or "Terminal"

Option B — Windows Command Prompt (CMD)

Classic terminal, slightly older but works fine for this project.

- Press `Windows` key + `R` (Run dialog opens).
- Type `cmd` and press `Enter`.
- A black window opens — this is your terminal.

Option C — Terminal Inside VS Code (Easiest for beginners)

If you install VS Code (see Section 4), it has a built-in terminal so you never need to leave the editor.

- Open VS Code.
- Go to the top menu: `View` → `Terminal` (or press `Ctrl + ``).
- A panel appears at the bottom of VS Code — this is your terminal.

❏ **Tip:**

All commands in this guide are written for PowerShell / CMD on Windows. If you are on macOS or Linux, use the `Terminal` app — the commands are identical except you may need to type `python3` instead of `python`.

4. Recommended Code Editors (IDEs)

You do not need a code editor just to run the application — but if you want to view or modify the source code, here are the best free options. They also provide the most convenient built-in terminals.

Editor	Best For
VS Code	Everything — Python, React, Git, terminal all in one. Best all-rounder.
PyCharm Community	Python / Flask backend focused. Excellent debugger.
WebStorm	JavaScript / React frontend focused.

✔ **Our Recommendation:**

Install Visual Studio Code (VS Code). It is free, runs on Windows / Mac / Linux, supports both Python and JavaScript out of the box, and has a built-in terminal that makes running this project much easier.

Useful VS Code Extensions to Install

After installing VS Code, open Extensions (Ctrl+Shift+X) and search for:

- Python (by Microsoft) — syntax highlighting and debugging for Flask
- ESLint — JavaScript / React code quality
- Prettier — automatic code formatting
- GitLens — visualise Git history

5. Cloning the Project from Git

"Cloning" means downloading a copy of the project code from a repository (e.g. GitHub or an internal Git server) to your computer. You only need to do this once.

5.1 Verify Git is Installed

Open a terminal and run:

```
git --version
```

You should see something like `git version 2.x.x`. If you see an error, download and install Git from <https://git-scm.com/downloads> first.

5.2 Clone the Repository

Navigate to the folder where you want to store the project, then run the clone command. Replace `<REPO_URL>` with the actual repository URL provided by the IT team.

```
# Navigate to your desired folder first (example: Documents)
cd C:\Users\YourName\Documents

# Clone the project
git clone <REPO_URL>

# Enter the project folder
cd SyrmaSGS_DashBoard
```

❏ **Example:**

If the repository URL is `https://github.com/company/SyrmaSGS_DashBoard.git`, then run:
`git clone https://github.com/company/SyrmaSGS_DashBoard.git`

5.3 Clone Using VS Code (No terminal needed)

If you prefer a graphical approach:

- Open VS Code.

- Press `Ctrl+Shift+P` to open the Command Palette.
- Type `Git: Clone` and press `Enter`.
- Paste the repository URL and choose a folder to save to.
- VS Code will download the project and offer to open it.

6. Backend Setup (Python / Flask)

The backend is the Python server that processes your SAP data files. Follow these steps in order. You only need to do Steps 6.1–6.3 once. Step 6.4 is what you run every day.

6.1 Verify Python is Installed

```
python --version
```

Expected output: Python 3.10.x (or any 3.10–3.13 version). If you see an error, download Python from <https://python.org/downloads> — make sure to tick "Add Python to PATH" during installation.

6.2 Navigate to the Project Root

All backend commands must be run from the root project folder (`SyrmaSGS_DashBoard`), NOT from a subfolder.

```
# Replace the path below with wherever you cloned the project
cd C:\Users\YourName\Documents\SyrmaSGS_DashBoard
```

6.3 Install Python Dependencies (One-time only)

This downloads all the Python libraries the backend needs (Flask, Pandas, NumPy, etc.).

```
pip install -r requirements.txt
```

This will download ~10–15 packages. It may take 1–3 minutes depending on your internet speed. You will see download progress lines. Wait until the command prompt returns.

☐ **If pip is not recognised:**

Run `python -m pip install -r requirements.txt` instead. If that also fails, close and reopen your terminal — Python may not have been added to PATH yet.

6.4 Start the Backend Server (Every time you use the app)

```
python app.py
```

You will see output like this — this means the server is running successfully:

```
* Serving Flask app 'backend.app'
* Debug mode: on
* Running on http://127.0.0.1:5000
* Running on http://192.168.x.x:5000
* Debugger PIN: xxx-xxx-xxx
```

```
Syrma Procurement Analytics -- Backend starting on http://localhost:5000
```

❑ **Important:**

Do NOT close this terminal window while using the dashboard. The backend must stay running. Open a SECOND terminal for the frontend.

7. Frontend Setup (Node.js / React)

The frontend is the visual dashboard that runs in your web browser. It is built with React and served locally by a tool called Vite. Open a SECOND terminal window for all commands in this section.

7.1 Verify Node.js is Installed

```
node --version  
npm --version
```

Expected output: v18.x.x or higher for Node, and 9.x.x or higher for npm. If you see an error, download Node.js from <https://nodejs.org> — choose the LTS version.

7.2 Navigate to the Frontend Folder

```
# From the project root:  
cd frontend  
  
# Or with the full path:  
cd C:\Users\YourName\Documents\SyrmaSGS_DashBoard\frontend
```

7.3 Install Node Packages (One-time only)

This downloads all JavaScript libraries (React, Recharts, Axios, etc.) into a local node_modules folder.

```
npm install
```

This may take 1–5 minutes. You will see progress output. Wait until the command prompt returns before continuing.

7.4 Start the Frontend Dev Server (Every time you use the app)

```
npm run dev
```

Successful output looks like this:

```
VITE v8.x ready in 200 ms  
  
→ Local: http://localhost:5173/  
→ Network: use --host to expose
```

Now open your web browser (Chrome or Edge recommended) and navigate to:

```
http://localhost:5173
```

✔ **Success check:**

You should see the Syrma SGS dark-themed dashboard in your browser. If it shows an error like "Cannot connect to backend", ensure the Python backend from Step 6.4 is still running in the other terminal.

8. Running the Application — Daily Usage

After the one-time setup in Sections 6 and 7 is complete, this is all you need to do every time you want to use the dashboard:

Terminal 1 — Start the Backend

```
# Navigate to the project root
cd C:\Users\YourName\Documents\SyrmaSGS_DashBoard

# Start the Flask server
python app.py

# You should see: Running on http://127.0.0.1:5000
# Leave this window open!
```

Terminal 2 — Start the Frontend

```
# Open a NEW terminal window, then:
cd C:\Users\YourName\Documents\SyrmaSGS_DashBoard\frontend

# Start the React dev server
npm run dev

# You should see: Local: http://localhost:5173/
# Leave this window open!
```

Browser

```
# Open Chrome or Edge and navigate to:
http://localhost:5173
```

❑ **To Stop the Application:**

Press `Ctrl + C` in each terminal window. This safely stops both servers.

9. Using the Dashboard — Step-by-Step

Once the application is running and you have opened <http://localhost:5173> in your browser:

Step 1: Upload Page

You are greeted with an upload zone. Drag-and-drop your SAP CSV / Excel files, or click the zone to browse. For the Procurement Dashboard, upload your ME2N (transactions) and EKKO (master data) files. For GR/IR Analytics, upload the GRIR, ME2N, and EKKO files.

Step 2: Merge / Process

After uploading, click "Merge Data". The backend will join and process your files. A loading indicator will appear — processing typically takes 5–30 seconds depending on file size.

Step 3: Procurement Dashboard (/dashboard)

View KPI summary cards (Total Spend, Open PO Value, Supplier Count), interactive charts (supplier analysis, monthly trends, currency exposure), and the AI chat assistant.

Step 4: GR/IR Dashboard (/grir)

Upload your GRIR file here for full reconciliation. Review open exposure, aging analysis, reconciliation status, and vendor rankings.

Step 5: GR/IR Analytics (/grir-analytics)

Upload GRIR + ME2N + EKKO together for a combined executive analytics view with KPI cards, chart visualisations, and management insights.

Step 6: Export

Use the Export buttons to download your analysis as CSV, Excel, or PDF for reporting.

10. AI Chat Assistant (Ollama — Optional)

The Procurement Dashboard includes a built-in AI chat assistant that can answer questions about your data (e.g. "Who is our top supplier?" or "What is the open PO value for Plant 1100?"). This requires Ollama — a free, locally-running AI tool. It is completely optional; the rest of the dashboard works without it.

10.1 Install Ollama

- Download Ollama from <https://ollama.com/download>
- Run the installer — no configuration needed.
- Ollama starts automatically in the background as a system service.

10.2 Pull an AI Model

Open a terminal and run one of the following commands. Choose based on your hardware (most company laptops have 16 GB RAM, CPU only):

Command	Model Size	Recommended For
ollama run llama3.2:3b	~2 GB	16 GB RAM laptop — best balance of speed and quality
ollama run phi3:mini	~2.2 GB	16 GB RAM laptop — excellent reasoning
ollama run qwen2.5:3b	~2 GB	16 GB RAM laptop — strong coding & analysis
ollama run tinyllama	~0.6 GB	Very slow/old machines or quick tests only
ollama run llama3.1:8b	~5 GB	Workstations with 32 GB RAM — best quality but slow

10.3 No Configuration Needed

The backend automatically detects which Ollama model you have installed. Simply start the backend (python app.py) and the chat assistant will use the available model. No environment variables or config files need to be edited for local use.

10.4 Using a Centralized GPU Server (IT Departments)

If Ollama is hosted on a shared server with a GPU (faster responses for all users):

```
# Windows PowerShell — set this before running python app.py
$env:OLLAMA_URL="http://<SERVER_IP>:11434/api/chat"
python app.py

# Example:
$env:OLLAMA_URL="http://192.168.1.50:11434/api/chat"
python app.py
```

11. Troubleshooting Common Errors

✗ "python" is not recognized as a command

Python is not installed or not added to PATH.

- Re-install Python from <https://python.org> — tick "Add Python to PATH" during setup.
- Then close and reopen your terminal.

✗ "npm" is not recognized as a command

Node.js is not installed or not added to PATH.

- Install Node.js from <https://nodejs.org> (choose LTS version).
- Close and reopen your terminal.

✗ "pip install" fails with permission error

- Run your terminal as Administrator (right-click → Run as Administrator).
- Or add --user flag: pip install --user -r requirements.txt

✗ Browser shows "Cannot connect to backend" or 502 error

The Flask backend is not running.

→ Open Terminal 1, navigate to the project root, and run: `python app.py`

→ Check that you see "Running on http://127.0.0.1:5000" in the output.

✗ Port 5000 already in use

→ Another application (or a previous Flask session) is using port 5000.

→ In PowerShell run: `Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess`

→ Then stop that process, or restart your computer.

✗ Port 5173 already in use

→ A previous npm run dev session is still running.

→ Press Ctrl+C in the frontend terminal, then run `npm run dev` again.

✗ git clone fails — "repository not found"

→ Verify you have network access to the repository server.

→ Check with IT for the correct repository URL and your access permissions.

✗ npm install produces EACCES permission errors

→ Run the terminal as Administrator.

→ Or delete the `node_modules` folder and try again: `rmdir /s /q node_modules` then `npm install`

✗ Dashboard loads but charts show "No Data"

→ You must upload your SAP files first using the Upload page.

→ After uploading, click the Merge button to process the data.

✗ AI Chat says "Could not connect to Ollama"

→ This is expected if you have not installed Ollama — the rest of the dashboard still works.

→ If you want the chat feature, install Ollama and pull a model (Section 10).

12. Quick-Reference Cheat Sheet

Keep this page handy. These are all the commands you need for daily use.

Task	Command / Action
Check Python version	<code>python --version</code>
Check Node version	<code>node --version</code>
Install Python packages	<code>pip install -r requirements.txt</code> (run once, from project root)
Install Node packages	<code>npm install</code> (run once, from frontend folder)
Start Flask backend	<code>python app.py</code> (from project root)
Start React frontend	<code>npm run dev</code> (from frontend folder)
Open the dashboard	Open browser → <code>http://localhost:5173</code>
Stop servers	Press Ctrl+C in each terminal window
Update code from Git	<code>git pull</code> (from project root)
Pull Ollama model	<code>ollama run llama3.2:3b</code>
Backend URL (API)	<code>http://localhost:5000/api/...</code>
Frontend URL	<code>http://localhost:5173</code>